# Achieved results: hybrid PQC on the StarFive VisionFive 2

Measured results from the FOCAL "Post-Quantum Retrofit" hackathon work: legacy vs hybrid post-quantum TLS 1.3 on a StarFive VisionFive 2 (JH7110, 4x SiFive U74 @ up to 1.5 GHz), OpenSSL 3.5.9.

- **Legacy:** X25519 key exchange (+ RSA-3072 server authentication)
- **Hybrid:** X25519MLKEM768 key exchange (+ ECDSA P-256 and ML-DSA-44 dual-signature authentication on our own server)

Two scenarios are measured throughout: 1. **Hackathon server** (IPv4), TLS handshake time, with server authentication RSA-3072 in both modes 2. **Own server** on the local network, end-to-end time including full hybrid authentication

Start with **SUMMARY.md** (PDF); sections 10, 11 and 12 cover the results in this folder.

## Folder layout: one folder per approach

Each approach is self-contained: a detailed report (`.md` + `.pdf`), all raw measurements, statistics, logs and the scripts used, so each can be written up on its own.

```
achieved_results/
├── README.md / README.pdf          this index
├── SUMMARY.md / SUMMARY.pdf        overall summary (sections 10–12 = these approaches)
├── baseline_default_clock/         BASELINE: default CPU setting, legacy vs hybrid
├── dynamic_cpu_freq/               APPROACH 1: raise the minimum CPU clock
├── bitmanip_zba_zbb/               APPROACH 2: RISC-V Zba/Zbb bit-manipulation extensions
├── multicore/                      APPROACH 3: classical and PQ halves on separate cores (in
progress)
└── tools/                          source code of clients, server, benchmarks and analysis
scripts
```

## Headline result

| Scenario | Baseline: legacy, default CPU | Hybrid, default CPU | Hybrid, min clock 750 MHz |
|---|---|---|---|
| Hackathon server, handshake (median) | 42.17 ms | 48.30 ms | **40.26 ms** |
| Own server, full hybrid auth, end-to-end (median) | 21.30 ms | 27.84 ms | **16.19 ms** |
| Idle temperature | 44.7 °C | 44.7 °C | **45.2 °C** |

Raising only the minimum CPU clock (`ondemand` governor, `scaling_min_freq` = 750 MHz) lets hybrid PQC beat the legacy baseline for +0.5 °C. Under identical CPU settings hybrid remains slightly slower than legacy (+0.5 ms on the LAN, +3–4.5 ms over the internet).

### RISC-V bit-manipulation extensions (Zba/Zbb)

Rebuilding OpenSSL with `-march=rv64gc_zba_zbb` (no code changes) speeds up SHA-3/SHAKE by 50 %, ML-KEM by 13–19 % and ML-DSA verification by 15 %, while classical algorithms are unchanged. End-to-end, full hybrid on the LAN drops from 28.15 to 26.57 ms (–5.6 %, faster in 20/20 blocks) at the default clock: a real gain, but about 7x smaller than the clock-policy change.

## Files

`SUMMARY.md`

Full project summary: setup, benchmarks, bottleneck analysis, migration guide, limitations. Section 10 = baseline at the default CPU clock; section 11 = dynamic CPU frequency, including the recommended setting. Sections 1–9 refer to earlier raw data that isn't included in this folder.

## `baseline_default_clock/`: BASELINE (default CPU setting: `ondemand`, min 312.5 MHz)

| File | Contents |
|---|---|
| `baseline_REPORT_default_clock.md` / `.pdf` | Report: legacy vs hybrid, 1000 runs per mode, both scenarios, best / median / percentiles / worst |
| `baseline_raw_1000.csv` | All 4000 measurements: `scenario,mode,block,run,ms` |
| `baseline_stats_1000.txt` | Computed statistics |
| `baseline_bench_1000.log` | Per-block log: governor, temperature, negotiated group / ML-DSA verification checks |
| `scripts/bench_1000.sh` | Benchmark script used (runs on the board) |

## `dynamic_cpu_freq/`: DYNAMIC CPU FREQ (raise only the minimum CPU clock)

| File | Contents |
|---|---|
| `dynamic_cpu_freq_REPORT.md` / `.pdf` | **Final report**: default vs `ondemand` + min 750 MHz, alternated block by block, 1000 runs per mode; includes the **RECOMMENDATION** section comparing all settings |
| `dynamic_cpu_freq_screen_REPORT.md` / `.pdf` | Screening of four settings (ondemand min 750 / 1250 MHz, schedutil min 312.5 / 750 MHz), 500 runs per mode |
| `dynamic_cpu_freq_confirm_raw.csv` | Confirmation run, all 8000 measurements: `config,scenario,mode,block,run,ms` (D0 = default, D1 = min 750 MHz) |
| `dynamic_cpu_freq_confirm_temps.csv` | Confirmation run temperatures: `config,phase,elapsed_s,temp_c` |
| `dynamic_cpu_freq_confirm_stats.txt` | Confirmation run statistics, incl. paired per-block comparison |
| `dynamic_cpu_freq_confirm.log` | Confirmation run per-block log and checks |
| `dynamic_cpu_freq_screen_raw.csv` | Screening run, all 8000 measurements (D1–D4) |
| `dynamic_cpu_freq_screen_temps.csv` | Screening run temperatures (idle + under load, every 5 s) |
| `dynamic_cpu_freq_screen_stats.txt` | Screening run statistics |
| `dynamic_cpu_freq_screen.log` | Screening run per-config log and checks |
| `scripts/bench_dyn.sh`, `scripts/bench_cfg_ab.sh` | Screening and alternating confirmation scripts (need sudo; restore the default CPU setting at the end) |

## `bitmanip_zba_zbb/`: BITMANIP (RISC-V Zba/Zbb bit-manipulation extensions)

| File | Contents |
|---|---|
| `bitmanip_REPORT.md` / `.pdf` | **Report**: plain `rv64gc` vs `-march=rv64gc_zba_zbb` OpenSSL build; micro-benchmarks and 1000-run end-to-end comparison vs the baseline |
| `bitmanip_micro_fixed1500.txt` | Crypto micro-benchmarks (RSA, ECDSA, ML-DSA, ML-KEM, X25519, SHA-3, SHA-256) for std / opt / bitmanip builds, 2 passes at a fixed 1.5 GHz |
| `bitmanip_micro_ondemand_UNRELIABLE.txt` | First attempt under the `ondemand` governor; clock noise made it inconsistent. Kept for transparency, not used |
| `bitmanip_e2e_raw.csv` | End-to-end, all 8000 measurements: `config,scenario,mode,block,run,ms` (config = plain / bitmanip) |
| `bitmanip_e2e_stats.txt` | Statistics incl. paired per-block comparison and the baseline for reference |
| `bitmanip_e2e.log` | Per-block log, library linkage of each binary, negotiated group / ML-DSA verification checks |
| `scripts/bench_bitmanip.sh`, `scripts/micro_bitmanip.sh`, `scripts/micro_fixed.sh` | End-to-end and micro-benchmark scripts |

### `tools/`: source code

| File | What |
|---|---|
| `hybrid_server.c` | TLS 1.3 test server: RSA-3072 for legacy clients; ECDSA P-256 + ML-DSA-44/65 dual signature bound to the TLS exporter for hybrid clients |
| `hybrid_client.c` | Benchmark client: pinned certificate + pinned ML-DSA key, end-to-end timing, `RAW=1` per-connection output, `TRACE=1` handshake timeline |
| `pqc_client.c` | Key-exchange client for the hackathon server (IPv4/IPv6, selectable group) |
| `sigbench.c` | Signature micro-benchmark (RSA, ECDSA, ML-DSA) |
| `analyze_1000.py`, `analyze_dyn.py`, `analyze_pair.py` | Statistics used for the reports |
| `md2pdf.py` | Markdown to PDF conversion used for the PDFs |

## Method notes

- Every individual measurement is recorded; modes (and, in the confirmation run, CPU settings) alternate in blocks of 50 so both see the same network conditions.
- Every hybrid block was checked: X25519MLKEM768 negotiated and, on the own server, the ML-DSA-44 signature verified.
- 0 failed connections across all runs in this folder.
- Bitmanip builds only run on RISC-V CPUs that implement Zba and Zbb (the U74 does).
- Not measured: energy (temperature used as the proxy); minimum clocks of 500 and 625 MHz.

---

Source: README.md